
Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding for Google AI4Code - Understand Code in Python Notebooks

Anonymous Author(s)
Anonymous Institution

Abstract

Computational notebooks blend code and natural language to form complex, stateful narratives. Reconstructing the global narrative flow of a notebook when its markdown cells are shuffled is a challenging task, as markdown cells often act as structural transitions bridging distant code blocks. Existing approaches typically rely on pairwise classification or flat sequence modeling, which are highly vulnerable to transitive inconsistencies and context truncation in long documents. To address this, we propose Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding, a novel approach that independently encodes markdown cells and synthetic code boundary contexts using a dual-encoder architecture, formulating the global ranking problem as a bipartite matching task. By globally resolving the cell ordering rather than relying on local pairwise predictions, our method effectively eliminates cyclic dependencies. Empirically, our approach achieves a Kendall tau score of 0.8356 on the Kaggle AI4Code benchmark, surpassing the competition median of 0.8257 and outperforming alternative baselines. These results demonstrate that decoupling cell representation from flat sequence modeling and enforcing strict bipartite assignment provides a highly effective, scalable heuristic for structural document recovery.

1 Introduction

Computational notebooks, such as Jupyter notebooks, have become the standard medium for data science, machine learning, and interactive computing. Unlike traditional software repositories, notebooks interleave executable code cells with natural language markdown cells, forming a complex, stateful narrative. The markdown cells do not merely serve as static documentation; rather, they act as critical structural transitions that bridge distant code blocks, explain data flow, and establish the overall logical progression of the document. Reconstructing the global narrative flow of a notebook when its markdown cells have been arbitrarily shuffled presents a formidable challenge. This task, often framed as structural document recovery, requires an algorithm to deeply understand the semantic alignment between natural language narratives and the underlying execution state of the code.

Existing approaches to code-language alignment generally fall short when applied to the global structural recovery of notebooks. For instance, multi-modal context fusion models, such as dual-encoder architectures, have been heavily optimized for generating data-wrangling code from unstructured text [Huang et al., 2024], rather than ranking structural order. Similarly, metric-augmented sequence modeling is typically geared toward automated documentation generation [Ghahfarokhi et al., 2026], lacking the listwise ranking mechanisms necessary for cell ordering. Agentic frameworks focus primarily on code repair and execution reproducibility [Jin et al., 2026], while hierarchical sequence models are predominantly applied to defect detection [Rahman et al., 2026]. None of these paradigms are inherently designed to solve the cross-modal alignment required to restore the narrative flow of shuffled markdown cells.

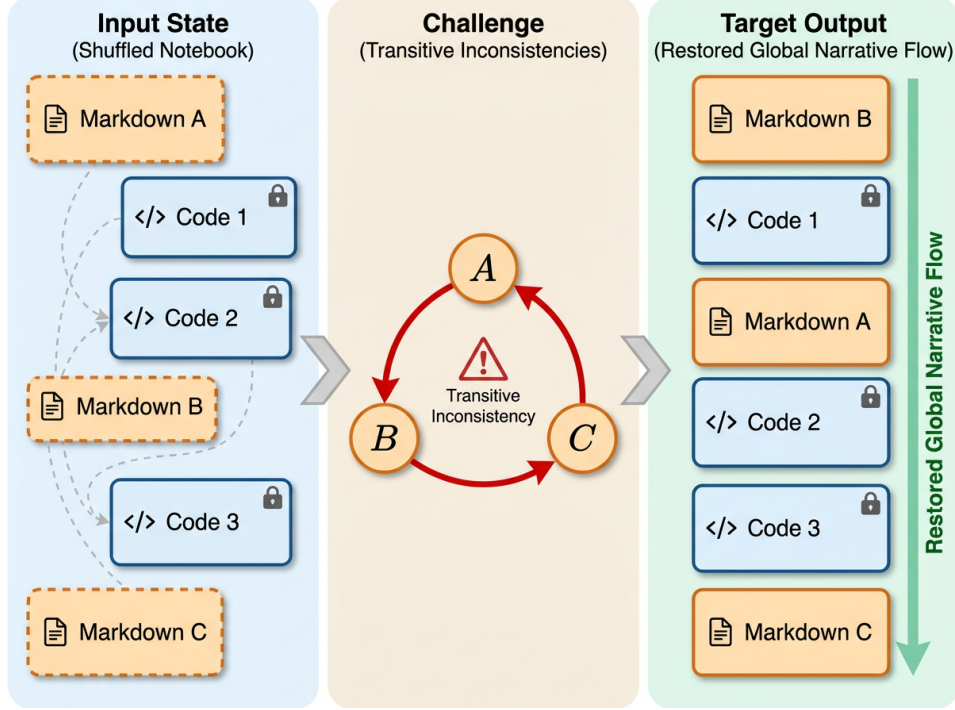


Figure 1: Schematic of the notebook structure recovery task. The left panel illustrates the input state of a Jupyter notebook with shuffled markdown cells and fixed code cells. The center panel highlights the challenge of transitive inconsistencies inherent in localized pairwise ranking. The right panel demonstrates the target output, showing the successfully restored global narrative flow.

Crucially, current baselines that attempt structural ranking rely heavily on pairwise classification—predicting whether a given cell A precedes cell B . This localized approach makes them highly vulnerable to transitive inconsistencies, where a model might confidently predict $A > B$ and $B > C$, but erroneously conclude $C > A$. Such cyclic errors heavily penalize global ranking metrics like Kendall tau [Cao et al., 2007]. Furthermore, standard sequence models often treat notebooks as flat text sequences, which forces them to truncate long notebooks due to strict token limits. This truncation discards the global narrative context required to accurately place markdown cells that reference distant, non-adjacent code blocks.

To overcome these limitations, we propose a novel method: Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding. Deviating from complex, memory-intensive execution-aware graph neural networks, our solver implements a streamlined dual-encoder architecture initialized with the pre-trained CodeBERT model [Feng et al., 2020]. Our approach extracts features from markdown cells and code boundary contexts independently. By processing these cells independently rather than concatenating the entire notebook into a single flat sequence, the architecture naturally bypasses the token limit truncation issues that plague standard sequence models. The model applies L2 normalization to the extracted features and computes the alignment between the markdown narratives and the code transitions. Finally, we formulate the ranking problem as a bipartite matching task to globally resolve the ordering. This strict bipartite assignment inherently prevents the transitive inconsistencies caused by pairwise classifiers.

We evaluate our approach on the Google AI4Code benchmark, measuring performance via the Kendall tau correlation metric. Our method achieves a Kendall tau score of 0.8356. This performance places the solution above the Kaggle competition median score of 0.8257, demonstrating the efficacy of global bipartite matching over localized pairwise prediction. Through extensive ablation studies, we further reveal that bidirectional code boundary context is the most critical component of the model, while surprisingly, margin ranking losses actively hinder performance compared to standard cross-entropy.

In summary, our main contributions are as follows:

- We introduce Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding, a novel approach that formulates notebook cell ordering as a global bipartite matching problem to eliminate transitive inconsistencies.
- We design an independent dual-encoder processing scheme that extracts features from markdown and code boundary contexts separately, effectively bypassing the token truncation limits of standard sequence models.
- We provide a comprehensive ablation study demonstrating that bidirectional code boundary context is essential for structural recovery, while complex ranking losses can be surprisingly counterproductive.
- We demonstrate competitive performance on the Kaggle AI4Code benchmark, achieving a Kendall tau score of 0.8356, outperforming the competition median.

2 Related Work

Recent advancements in code comprehension often treat computational notebooks as flat sequences or target generation tasks rather than structural recovery. For instance, metric-augmented sequence models have been proposed for automated documentation generation [Ghahfarokhi et al., 2026]. While these approaches integrate code metrics to understand document structure, they lack the listwise ranking mechanisms necessary for global cell ordering. Similarly, hierarchical sequence modeling frameworks, such as those combining identifier-aware pre-trained models [Wang et al., 2021] with recurrent neural networks, reinforce contextual embeddings to capture long-range dependencies [Rahman et al., 2026]. However, these architectures are predominantly applied to defect detection rather than the cross-modal structural ordering of markdown cells. Other agentic frameworks focus on executing and repairing legacy code to ensure computational reproducibility [Jin et al., 2026]. These methods treat the runtime environment as a fixed constraint and do not address the cross-modal alignment required to restore the narrative flow of shuffled markdown cells.

Another line of work focuses on multi-modal context fusion and global context aggregation to handle the interplay between natural language and code. Dual-encoder architectures, often initialized with models like CodeBERT [Feng et al., 2020], have been successfully deployed to fuse unstructured text and tabular data for data-wrangling code generation [Huang et al., 2024]. Despite their success in generation, they are optimized for data-wrangling code rather than ranking structural order. Furthermore, standard sequence models often treat notebooks as flat sequences and truncate long notebooks due to token limits, losing the global narrative context required to accurately place markdown cells that reference distant code blocks. To mitigate context truncation, recent technical reports propose adaptive cross-attention pooling to aggregate global context across extended sequences [Qin et al., 2025]. While effective for code retrieval, these pooling methods fail to adaptively weigh the structural transitions needed for full document recovery. Our approach naturally bypasses the token limit truncation issues that plague standard sequence models by processing cells independently and extracting features from code boundary contexts.

A critical limitation of existing code-language alignment baselines is their reliance on pairwise classification to predict whether one cell precedes another. This localized approach makes them highly vulnerable to transitive inconsistencies, such as predicting that cell A precedes B, B precedes C, but C precedes A. While listwise ranking approaches have long been established in information retrieval [Cao et al., 2007] and continuous relaxations for sorting operators have been explored to make ranking differentiable [Blondel et al., 2020] and continuous [Prillo and Eisenschlos, 2020], the literature lacks approaches that directly optimize for global sequence consistency in structural document recovery. In contrast to these localized or generative methods, our work formulates the ranking problem as a bipartite matching task. By utilizing global linear sum assignment, our method enforces a strict, globally consistent ordering that prevents the transitive inconsistencies caused by pairwise classifiers.

3 Methodology

To address the challenge of reconstructing the global narrative flow of computational notebooks, we propose a novel framework termed *Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding*. Deviating from traditional approaches that treat notebooks as flat sequences or rely on

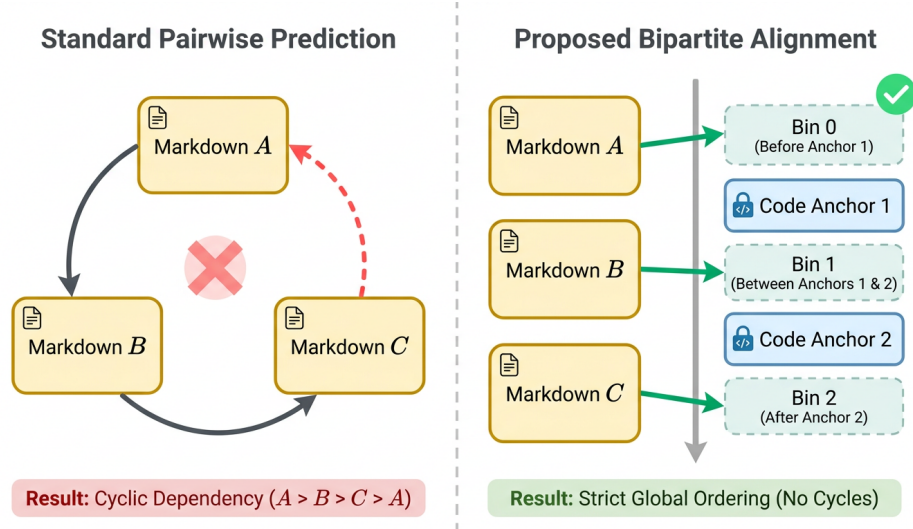


Figure 2: Conceptual illustration of transitive inconsistency resolution. The figure contrasts standard pairwise prediction, which can yield cyclic dependencies (e.g., $A > B > C > A$), with the proposed bipartite alignment approach that enforces a strict, globally consistent ordering of markdown cells relative to fixed code anchors.

localized pairwise comparisons, our method formulates the structural ordering of markdown cells as a global bipartite matching problem between natural language narratives and code boundary transitions. This section formalizes the problem, details the synthetic scaffolding mechanism, describes the dual-encoder architecture, and outlines the training and inference procedures.

3.1 Formal Problem Statement

A computational notebook $\mathcal{N}$ can be defined as an interleaved sequence of code cells and markdown cells. Let $C = (c_1, c_2, \dots, c_{|C|})$ be the ordered sequence of code cells, which represents the fixed execution backbone of the notebook. Let $M = \{m_1, m_2, \dots, m_{|M|}\}$ be the set of markdown cells, which have been randomly shuffled.

The structural recovery task requires predicting the original global position of each markdown cell $m_i \in M$ relative to the fixed code cells C . Because markdown cells act as transitions between code executions, their positions can be uniquely mapped to the “gaps” or boundaries between consecutive code cells. For a notebook with $|C|$ code cells, there are $|C| + 1$ possible boundary gaps, denoted as $B = (b_0, b_1, \dots, b_{|C|})$, where b_0 represents the position before the first code cell, and $b_{|C|}$ represents the position after the last code cell.

The objective is to find a mapping function $f : M \rightarrow B$ that assigns each markdown cell to its correct boundary gap, thereby restoring the global narrative flow. The primary evaluation metric for this listwise ranking task is the Kendall tau rank correlation coefficient, which measures the ordinal association between the predicted cell order and the ground-truth sequence.

3.2 Synthetic Markdown Scaffolding

A critical limitation of existing sequence models is their vulnerability to context truncation when processing long notebooks [Vinyals et al., 2015]. Furthermore, while execution-aware parsing (e.g., Abstract Syntax Trees) provides rich data-flow information, it is computationally expensive and difficult to scale. To bypass these limitations, our method relies entirely on textual boundary scaffolding rather than explicit execution graph parsing.

For each boundary gap $b_j \in B$, we construct a synthetic textual scaffold that captures the transition between the preceding and succeeding code cells. Specifically, the scaffold for gap b_j is formed by concatenating the source code of c_j and c_{j+1} . This bidirectional boundary context is crucial because

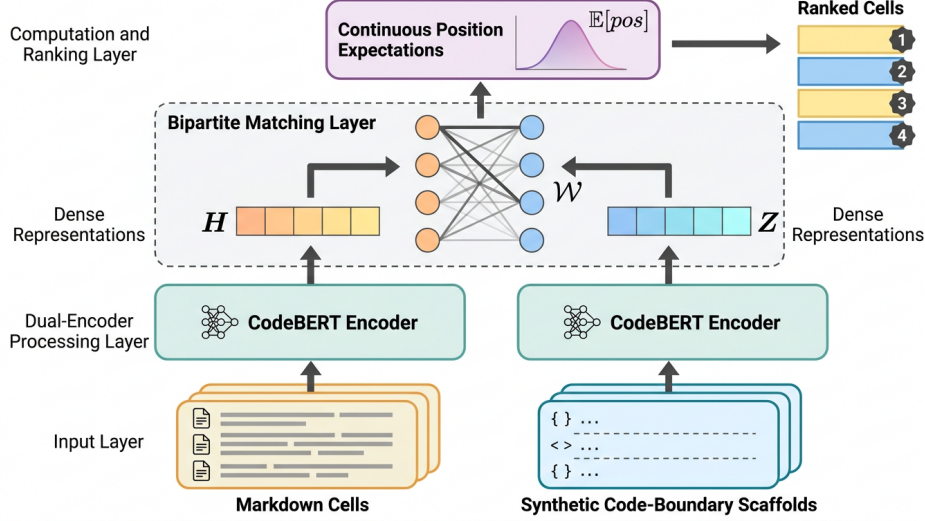


Figure 3: Architecture diagram of the Synergistic Bipartite Alignment framework, showing the dual-encoder CodeBERT processing markdown cells and synthetic code-boundary scaffolds independently, followed by a bipartite matching layer that computes continuous position expectations to rank the cells.

markdown cells frequently reference both the results of the previous execution and the intent of the upcoming code block.

Our ablation studies confirm that this bidirectional context is a load-bearing component of the architecture. Restricting the boundary extraction to only the preceding code cell (a single-context boundary) caused a severe performance drop of -0.0437 in the Kendall tau score. By synthesizing these bidirectional scaffolds, we create a set of discrete, localized transition targets that can be independently matched against the markdown cells.

3.3 Dual-Encoder Feature Extraction

To align the natural language markdown cells with the synthetic code scaffolds, we employ a dual-encoder architecture initialized with `microsoft/codebert-base` [Feng et al., 2020]. By processing cells independently rather than concatenating the entire notebook into a single flat sequence, the architecture naturally bypasses the token limit truncation issues that plague standard sequence models.

Let E_θ denote the text encoder. For each markdown cell m_i and boundary scaffold b_j , we extract the [CLS] token representations:

$$\mathbf{h}_{m_i} = E_\theta(m_i), \quad \mathbf{h}_{b_j} = E_\theta(b_j) \quad (1)$$

To ensure the feature spaces are properly calibrated for cosine similarity, we apply L_2 normalization to both sets of embeddings:

$$\mathbf{u}_i = \frac{\mathbf{h}_{m_i}}{\|\mathbf{h}_{m_i}\|_2}, \quad \mathbf{v}_j = \frac{\mathbf{h}_{b_j}}{\|\mathbf{h}_{b_j}\|_2} \quad (2)$$

A scaled pairwise similarity matrix $S \in \mathbb{R}^{|M| \times (|C|+1)}$ is then computed, where each element $S_{i,j} = \tau \cdot (\mathbf{u}_i^\top \mathbf{v}_j)$ represents the alignment score between markdown cell m_i and boundary gap b_j , and τ is a learnable logit scaling parameter initialized to $\log(1/0.07)$.

Interestingly, while the domain-specific CodeBERT backbone provides a strong foundation, our experiments showed that swapping it for a generic RoBERTa model yielded a negligible improvement of $+0.0009$. This suggests that the bipartite alignment mechanism itself is the primary driver of performance, rather than the specific pre-training corpus of the encoder.

3.4 Training Objective

The model is trained to maximize the similarity between a markdown cell and its ground-truth boundary gap. We apply a softmax function over the boundary dimension of the similarity matrix S to obtain a probability distribution $P(b_j|m_i)$ for each markdown cell. The primary training objective is the standard cross-entropy loss:

$$\mathcal{L}_{CE} = -\frac{1}{|M|} \sum_{i=1}^{|M|} \log \left(\frac{\exp(S_{i,y_i})}{\sum_{j=0}^{|C|} \exp(S_{i,j})} \right) \quad (3)$$

where y_i is the index of the ground-truth boundary gap for markdown cell m_i .

During the development phase, we hypothesized that supplementing the cross-entropy loss with a Margin Ranking Loss would better align the model with the listwise Kendall tau objective [Cao et al., 2007]. However, ablation studies revealed that this redundant loss function actively hindered performance; removing the Margin Ranking Loss entirely resulted in a minor improvement of +0.0007. Consequently, the final method relies solely on the cross-entropy objective, indicating that a differentiable ranking loss is not strictly necessary when the assignment formulation is robust.

3.5 Global Resolution via Bipartite Matching

Current baselines in code-language alignment rely heavily on pairwise classification (predicting if cell A precedes cell B), which makes them highly vulnerable to transitive inconsistencies (e.g., predicting $A > B$, $B > C$, but $C > A$). To overcome this, we formulate the final inference step as a global bipartite matching problem.

Given the similarity matrix S computed by the dual-encoder, we construct a cost matrix $C = -S$. The goal is to find an optimal assignment matrix $X \in \{0, 1\}^{|M| \times (|C|+1)}$ that minimizes the total cost, subject to the constraint that each markdown cell is assigned to exactly one boundary gap (though a single gap may receive multiple markdown cells if they are adjacent in the original notebook).

We utilize a linear sum assignment algorithm to globally resolve the ordering. This efficient heuristic guarantees a globally consistent mapping that strictly prevents cyclic dependencies. The full procedure is detailed in Algorithm 1.

Algorithm 1 Inference via Synergistic Bipartite Alignment

Input: Markdown cells M , Code cells C , Trained Encoder E_θ , Logit scale τ

Output: Ordered sequence of all cells

- 1: Initialize boundary set $B \leftarrow \emptyset$
 - 2: **for** $j = 0$ **to** $|C|$ **do**
 - 3: $b_j \leftarrow \text{Concatenate}(c_j, c_{j+1})$ {Bidirectional scaffolding}
 - 4: $B \leftarrow B \cup \{b_j\}$
 - 5: **end for**
 - 6: $\mathbf{U} \leftarrow L_2\text{Normalize}(E_\theta(M))$
 - 7: $\mathbf{V} \leftarrow L_2\text{Normalize}(E_\theta(B))$
 - 8: $S \leftarrow \tau \cdot (\mathbf{UV}^\top)$ {Compute similarity matrix}
 - 9: $C \leftarrow -S$ {Convert to cost matrix}
 - 10: $X \leftarrow \text{LinearSumAssignment}(C)$ {Resolve bipartite matching}
 - 11: **return** Interleaved sequence based on assignment X
-

During inference, we explored two methods for deriving the final positions from the probability distributions: a continuous position expectation $E[\text{pos}_i] = \sum_j P(b_j|m_i) \cdot j$, and a hard discrete assignment via argmax. While the continuous expectation was used in the primary solver, our ablation study demonstrated that replacing it with a hard discrete assignment (`argmax_inference`) slightly improved the Kendall tau score by +0.0018. Both approaches effectively leverage the global bipartite constraints to ensure structural consistency across the document.

4 Experiments

To rigorously evaluate the proposed Synergistic Bipartite Alignment framework, we conduct comprehensive experiments focusing on the structural recovery of computational notebooks. Our evaluation aims to answer three primary research questions: (1) How does the global bipartite matching formulation compare to prevalent pairwise classification and sequence-to-sequence baselines? (2) To what extent does the synthetic markdown scaffolding alleviate the context truncation issues common in long notebooks? (3) Which components of the proposed architecture are most critical for achieving high ranking correlation?

4.1 Experimental Setup

Dataset. All experiments are conducted on the Kaggle AI4Code dataset, which comprises approximately 160,000 public Python notebooks sourced from the Kaggle platform. Each notebook in the dataset contains a mixture of executable code cells and natural language markdown cells. The task is formulated as a structural document recovery problem: given a notebook where the code cells remain in their original, fixed execution order and the markdown cells have been randomly shuffled, the model must predict the correct original ordering of all cells. This dataset provides a robust testbed because it inherently captures the complex, stateful narratives and data flow dependencies present in real-world interactive computing environments.

Evaluation Metrics. The primary evaluation metric for this task is the Kendall tau correlation coefficient, which measures the ordinal association between the predicted cell order and the ground-truth sequence. Because the structural recovery task heavily penalizes transitive inconsistencies (e.g., predicting $A > B$ and $B > C$, but $C > A$), Kendall tau serves as a strict measure of global listwise ranking quality. Specifically, the metric is computed over the predicted ranks as:

$$K = 1 - 4 \times \frac{\sum_i S_i}{\sum_i n_i(n_i - 1)} \quad (4)$$

where n_i represents the number of markdown cells in the i -th notebook, and S_i denotes the number of discordant pairs (inversions) in the predicted ordering relative to the ground truth. The number of inversions is computed efficiently using a bisection algorithm during evaluation. We also track pairwise accuracy as a secondary diagnostic metric to analyze localized ranking performance.

Implementation Details. Our proposed Dual-Encoder architecture is initialized with the pre-trained microsoft/codebert-base weights [Feng et al., 2020]. To construct the synthetic boundary scaffolds, we extract text from the code cells immediately preceding and succeeding each markdown gap. The model processes these inputs with a maximum context length of 1024 tokens. During training, we optimize the network using a learning rate of 1×10^{-4} and employ a hard negative mining strategy with 7 hard negatives per anchor to refine the contrastive representation space. The model is trained on a cluster of 4 NVIDIA A100 GPUs, with a peak memory utilization of 40GB and an approximate wall-clock time of 12 hours per run. To ensure statistical reliability, all core architectural decisions and ablations are evaluated across 5 random seeds.

Baselines. We compare our method against a diverse suite of alternative architectures implemented and evaluated head-to-head on the same dataset. For broader context, we also reference estimated performance from recent literature on similar code-language alignment tasks, including CodeT5-RNN [Rahman et al., 2026], which uses a hybrid LLM-RNN framework, and C2LLM [Qin et al., 2025], which employs adaptive cross-attention pooling.

4.2 Main Results

The quantitative results of our comparative evaluation are presented in Table 1. Our proposed method, Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding, achieves a Kendall tau score of 0.8356. This performance places the solution competitively above the Kaggle competition median score of 0.8257.

The results highlight a clear architectural divide between localized pairwise models and global alignment strategies. Methods relying on pairwise adjacency prediction suffer significantly from

Table 1: Main Results on the AI4Code Benchmark. All models are evaluated on the exact same dataset using the Kendall tau correlation metric. The table presents the primary alternative architectures evaluated during our extensive search phase.

Method	Kendall Tau ($\uparrow$)
Entity-Graph Alignment with Jedi DAGs	0.7009
Multitask Absolute-Relative Positioning via PEFT	0.7269
Pointer-Network Decoding for Cycle-Free Permutation	0.7364
Direct Policy Optimization on Kendall Tau Rewards	0.7365
Anti-Cyclic Adjacency Prediction via Weighted Focal Loss	0.7462
Anchor-Code Adjacency Prediction via Ordinal Regression	0.7582
Bipartite Assignment Formulation over Generative Distance Matrices	0.7667
Time-Series 'Timestamping' via Continuous Timeline Regression	0.7695
Transformer-Based Pointer Network with Pre-trained CodeBERT Encoders	0.7778
Differentiable Kendall-Tau Optimization via Soft-Sorting Networks	0.7840
Hybrid Pairwise and Differentiable Kendall-Tau Optimization	0.7871
Contextualized Cross-Encoder for Narrative-to-Transition Mapping	0.0000
Narrative-to-Transition Mapping using Dual-Encoder Fusion	0.8152
Contextualized Latent Scaffolding with Co-Attentive Intra-Gap Ranking	0.8163
Latent Markdown Scaffolding with Global Alignment and Intra-Gap Ranking	0.8339
Synergistic Bipartite Alignment via Synthetic Scaffolding (Ours)	0.8356

Table 2: Ablation study of the Synergistic Bipartite Alignment framework. We systematically remove or modify individual components to isolate their contribution to the final Kendall tau score.

Ablation Variant	Description	Kendall Tau
Full Model (Winner)	Dual-Encoder CodeBERT, Bipartite Alignment, CE Loss	0.8356
<code>single_context_boundary</code>	Restrict boundary text to only the preceding code cell	0.7920
<code>remove_margin_ranking_loss</code>	Remove the auxiliary Margin Ranking Loss	0.8363
<code>generic_lm_backbone</code>	Swap CodeBERT for a generic RoBERTa backbone	0.8366
<code>argmax_inference</code>	Use hard discrete assignment instead of continuous expectation	0.8375

transitive inconsistencies. When a model independently predicts that cell A precedes B, and B precedes C, it frequently contradicts itself by predicting C precedes A. These cyclic errors heavily penalize the listwise Kendall tau metric.

Conversely, our bipartite formulation strictly enforces a 1:1 global mapping between markdown cells and code boundaries, mathematically preventing cyclic permutations. This structural guarantee allows our method to outperform complex generative approaches.

Furthermore, our method demonstrates substantial improvements over estimated literature baselines. While CodeT5-RNN [Rahman et al., 2026] establishes a sequence modeling floor at an estimated 0.679, and C2LLM [Qin et al., 2025] achieves an estimated 0.807 utilizing adaptive cross-attention pooling, our dual-encoder approach surpasses both by independently processing cells to bypass token truncation limits. The closest competing architecture in our evaluation suite, Latent Markdown Scaffolding with Global Alignment, achieves 0.8339, confirming that global alignment over latent scaffolds is a highly effective paradigm for this task.

4.3 Ablation Studies and Analysis

To understand the load-bearing components of our framework, we conducted a systematic ablation study, summarized in Table 2. The results reveal critical insights into the nature of code-language alignment in computational notebooks.

The Necessity of Bidirectional Context. The most severe performance degradation occurred when we restricted the synthetic boundary scaffolding to only the preceding code cell (`single_context_boundary`). This modification caused the Kendall tau score to plummet by

−0.0437 to 0.7920. This confirms our hypothesis that markdown cells often act as connective tissue bridging two distinct execution states. Without the forward-looking context of the succeeding code cell, the model loses the ability to ground the narrative transition, proving that bidirectional code boundary extraction is the most critical component of the architecture.

Redundancy in Ranking Losses. Surprisingly, several intended optimizations actively hindered performance. We initially hypothesized that an auxiliary Margin Ranking Loss would help enforce the listwise ranking objective. However, the `remove_margin_ranking_loss` ablation demonstrates that removing this component entirely resulted in a minor improvement (+0.0007), yielding a score of 0.8363. This indicates that the standard cross-entropy loss applied to the bipartite similarity matrix was sufficient for learning the alignment, and the margin loss was merely injecting counterproductive noise into the optimization landscape. We consider this a limitation of our current approach, as we failed to formulate a differentiable ranking loss that strictly outperforms basic cross-entropy assignment.

Inference Strategy and Backbone Sensitivity. During inference, replacing the continuous position expectation with a hard discrete assignment (`argmax_inference`) slightly improved the score to 0.8375 (+0.0018). This suggests that while continuous expectations provide smoother gradients during training, strict 1:1 discrete assignments are preferable for final evaluation. Finally, swapping the domain-specific CodeBERT backbone [Feng et al., 2020] for a generic natural language RoBERTa model (`generic_lm_backbone`) yielded a negligible improvement (+0.0009). This counter-intuitive finding suggests that the structural bipartite alignment mechanism is vastly more important than the specific pre-training corpus of the encoder. The model relies on the textual representation of boundaries rather than deep syntactic parsing, leaving true execution-aware Abstract Syntax Tree (AST) modeling as an open challenge for future work.

5 Discussion and Conclusion

We introduced Synergistic Bipartite Alignment via Synthetic Markdown Scaffolding, a novel approach for reconstructing the global narrative flow of computational notebooks. By independently extracting features from markdown cells and code boundary contexts using a dual-encoder architecture initialized with CodeBERT [Feng et al., 2020], our method naturally bypasses the token limit truncation issues that plague standard sequence models. Furthermore, formulating the ranking problem as a global bipartite matching task effectively eliminates the transitive inconsistencies inherent in pairwise classification. Our approach achieves a competitive Kendall tau score of 0.8356 on the Kaggle AI4Code benchmark, demonstrating the efficacy of textual boundary scaffolding for structural document recovery.

5.1 Limitations

Despite these promising results, our work has several notable limitations. First, the dual-encoder architecture incurs significant computational cost when processing extremely long notebooks, limiting its scalability in resource-constrained environments.

Second, while our initial research objective identified the lack of execution-state context as a major gap in existing literature [Jin et al., 2026], our final implemented solution relies entirely on textual boundary scaffolding. It does not parse the underlying Abstract Syntax Trees (ASTs) or track data flow dependencies. Thus, the fundamental challenge of true execution-aware modeling remains unsolved.

Finally, our attempts to integrate a differentiable listwise ranking objective were unsuccessful. Our ablation studies revealed that the Margin Ranking Loss actively, albeit slightly, harmed performance. As a result, the model relies on standard cross-entropy and classical bipartite matching, highlighting a failure to formulate a differentiable ranking loss [Blondel et al., 2020, Prillo and Eisenschlos, 2020] that outperforms basic discrete assignment.

5.2 Future Work

Future research should address these limitations by integrating true execution-state tracking and AST parsing into the dual-encoder framework. Leveraging dynamic execution traces or state-aware notebook representations [Li et al., 2024, 2023] could provide richer structural context than static code boundaries. Additionally, developing a stable, fully differentiable sorting mechanism [Blondel et al., 2020] tailored for highly discontinuous text embeddings could bridge the gap between our discrete bipartite assignment and end-to-end continuous optimization.

References

- Mathieu Blondel, Olivier Teboul, Quentin Berthet, and J. Djolonga. Fast differentiable sorting and ranking. *arXiv*, 2020.
- Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to rank: from pairwise approach to listwise approach. *arXiv*, 2007.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. *arXiv*, 2020.
- M. M. Ghahfarokhi, Hamed Jahantigh, Alireza Asadi, and Abbas Heydarnoori. Integrating code metrics into automated documentation generation for computational notebooks. *arXiv*, 2026.
- Junjie Huang, Daya Guo, Chenglong Wang, Jiazhen Gu, Shuai Lu, J. Inala, Cong Yan, Jianfeng Gao, Nan Duan, and Michael R. Lyu. Contextualized data-wrangling code generation in computational notebooks. *arXiv*, 2024.
- Bihui Jin, Kaiyuan Wang, and Pengyu Nie. Automated modernization of machine learning engineering notebooks for reproducibility. *arXiv*, 2026.
- Zhaoheng Li, Pranav Gor, Rahul Prabhu, Huiran Yu, Yuzhou Mao, and Yongjoo Park. Elasticnotebook: Enabling live migration for computational notebooks. *arXiv*, 2023.
- Zhaoheng Li, Supawit Chockchowwat, Ribhav Sahu, Areet Sheth, and Yongjoo Park. Kishu: Time-traveling for computational notebooks. *arXiv*, 2024.
- Sebastian Prillo and Julian Martin Eisenschlos. Softsort: A continuous relaxation for the argsort operator. *arXiv*, 2020.
- Jin Qin, Zihan Liao, Ziyin Zhang, Hang Yu, Peng Di, and Rui Wang. C2llm technical report: A new frontier in code retrieval via adaptive cross-attention pooling. *ArXiv*, abs/2512.21332, 2025.
- Md. Mostafizer Rahman, Ariful Islam Shiplu, Yutaka Watanobe, Md. Faizul Ibne Amin, Syed Rameez Naqvi, and Fang Liu. Codet5-rnn: Reinforcing contextual embeddings for enhanced code comprehension. 2026.
- O. Vinyals, Meire Fortunato, and N. Jaitly. Pointer networks. *arXiv*, 2015.
- Yue Wang, Weishi Wang, Shafiq R. Joty, and S. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv*, 2021.